

IF2224 - Teori Bahasa Formal dan Otomata

Tugas Besar

Milestone 4 : Intermediate Code and Interpreter

21 Mei 2026 - 6 Juni 2026



Mirza Tsabita Wafa'ana 13524114

Nathanael Shane Bennet 13524119

Faris Wirakusuma Triawan 13524130

Jonathan Harijadi 13524144

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

BAB 1 : Teori Singkat.....	3
1.1. Intermediate Code.....	3
1.2. Interpreter.....	3
BAB 2 : Perancangan & Implementasi.....	4
2.1. Implementasi Program.....	4
A. Pembangkitan Intermediate Code (IC Generator).....	4
B. Manajemen Memori Runtime (StackMemory).....	5
C. Interpreter Berbasis Mesin Tumpukan (SafeStackMachine / VirtualMachine).....	5
BAB 3 : Pengujian.....	8
BAB 4 : Kesimpulan dan Saran.....	13
4.1. Kesimpulan.....	13
4.2. Saran.....	13
Lampiran.....	14
Referensi.....	15

BAB 1 : Teori Singkat

1.1. Intermediate Code

Intermediate code merupakan representasi penengah dari source code sebelum dikonversi menjadi bahasa mesin atau dieksekusi. Tujuannya adalah memisahkan arsitektur front-end (analisis leksikal dan sintaksis) dari back-end (eksekusi atau code generation). Format yang umum digunakan adalah *Three-Address Code* (TAC), bytecode, atau representasi instruksi langsung pada *Abstract Syntax Tree* (AST).

1.2. Interpreter

Interpreter adalah program perangkat lunak yang mengeksekusi instruksi langsung dari representasi intermediate code atau secara rekursif mengevaluasi AST (*tree-walk interpreter*). Berbeda dengan kompilator murni yang menghasilkan *executable binary*, interpreter memerlukan tabel simbol (*symbol table*) pada saat *runtime* untuk mengelola alokasi memori, pemetaan variabel, dan evaluasi ekspresi secara dinamis.

Interpreter bekerja dengan prinsip eksekusi instruksi secara berurutan. Setiap instruksi dibaca, dikenali jenis operasinya, lalu dieksekusi oleh mesin virtual. Konsep ini sejalan dengan penjelasan Nystrom dalam *Crafting Interpreters* bahwa virtual machine merupakan bagian dari arsitektur interpreter yang menerima sekumpulan bytecode atau instruksi dan menjalankannya melalui fungsi eksekusi utama [1]. Selama proses eksekusi, interpreter menggunakan instruction pointer atau program counter untuk menentukan instruksi yang sedang dijalankan. Instruction pointer akan bergerak ke instruksi berikutnya setelah satu instruksi selesai dieksekusi, kecuali pada instruksi lompatan seperti JMP atau JPC [1].

Pada program yang dikembangkan, interpreter menggunakan model Stack Machine. Stack Machine adalah model eksekusi yang menggunakan stack untuk menyimpan nilai sementara selama proses perhitungan. Instruksi LIT digunakan untuk memasukkan nilai literal ke stack, LOD digunakan untuk mengambil nilai dari memori, STO digunakan untuk menyimpan nilai ke memori, sedangkan OPR digunakan untuk menjalankan operasi aritmatika atau logika. Model ini sesuai untuk evaluasi ekspresi karena nilai operand dapat dimasukkan ke stack terlebih dahulu, kemudian operator mengambil nilai tersebut dari stack untuk menghasilkan nilai baru [2].

BAB 2 : Perancangan & Implementasi

2.1. Implementasi Program

Pada *Milestone* ini, program memisahkan secara tegas arsitektur *front-end* dan *back-end* melalui pembangkitan Intermediate Code (IC) dan mesin pengeksekusi (Interpreter/Virtual Machine). Implementasi ini dibangun menggunakan C++ yang terbagi menjadi tiga modul utama: pembangkit kode (*IC Generator*), manajemen memori (*Stack Memory*), dan mesin pengeksekusi yang aman (*SafeStackMachine* dan *VirtualMachine*).

A. Pembangkitan Intermediate Code (IC Generator)

Pembangkitan kode diimplementasikan menggunakan pola desain *Visitor* pada kelas *ICGenerator* yang menelusuri (*traverse*) struktur pohon sintaks abstrak (AST). Kode yang dihasilkan berbentuk *Three-Address Code* (TAC) atau P-Code yang dieksekusi secara sekuensial. Mekanisme seperti *backpatching* digunakan untuk menangani lompatan (*jump*) pada struktur alur kontrol (*IF*, *WHILE*, *FOR*).

Cuplikan Kode: Implementasi fungsi evaluasi operasi biner dan *backpatching* pada ekspresi *IF*:

```
void ICGenerator::visit(BinOpNode* node) {
    if (node->left) node->left->accept(this);
    if (node->right) node->right->accept(this);

    if (node->op == "+") emit("OPR", 0, 2);
    else if (node->op == "-") emit("OPR", 0, 3);
    else if (node->op == "*") emit("OPR", 0, 4);
    // ... dst

void ICGenerator::visit(IfNode* node) {
    if (node->condition) node->condition->accept(this);
    int jmpIdx = emit("JPC", 0, -1);
    if (node->thenBranch) node->thenBranch->accept(this);
    if (node->elseBranch) {
        int jmpIdx = emit("JMP", 0, -1);
        backpatch(jmpIdx, currentLine);
        node->elseBranch->accept(this);
    }
```

```

        backpatch(jmpIdx, currentLine);
    } else {
        backpatch(jpcIdx, currentLine);
    }
}

```

B. Manajemen Memori Runtime (StackMemory)

Eksekusi program secara independen dikendalikan oleh struktur memori berbasis *stack* (**StackMemory**). Kelas ini mensimulasikan lingkungan alokasi memori dinamis yang melakukan pencatatan indeks (menggunakan **stackPointer** dan **basePointer**) serta menerapkan pengecekan terhadap anomali alokasi seperti *overflow* dan *underflow*.

Cuplikan Kode: Penanganan validasi memori pada kelas **StackMemory**:

```

void StackMemory::checkOverflow(int increment) const {
    if (stackPointer + increment >= MAX_STACK_SIZE) {
        throw std::runtime_error("Runtime Error: Stack Overflow. Batas alokasi memori runtime Arion terlampaui.");
    }
}

void StackMemory::push(int value) {
    checkOverflow(1);
    stackPointer++;
    memory[stackPointer] = value;
}

```

C. Interpreter Berbasis Mesin Tumpukan (SafeStackMachine / VirtualMachine)

Interpreter mengeksekusi langsung instruksi *Intermediate Code* melalui iterasi yang membaca *OpCode* (seperti **LOD**, **STO**, **JMP**, **OPR**). Guna menjamin kestabilan program, eksekusi dilakukan dalam modul **SafeStackMachine** yang bekerja berdampingan dengan **RuntimeProtection** untuk mencegah *crash* atau *segmentation fault* pada level sistem operasi saat menjumpai komputasi ilegal seperti pembagian dengan nol atau instruksi lompat yang tidak valid.

Cuplikan Kode: Siklus eksekusi utama pada **SafeStackMachine** dengan penyertaan lapisan proteksi:

```

void SafeStackMachine::run() {
    while (!halted) {
        RuntimeProtection::checkInstructionPointer(ip, code.size());
        const Instruction instruction = code[ip];

        switch (instruction.op) {
            case OpCode::INT: {
                if (instruction.operand < 0) {
                    throw RuntimeError("MemoryAllocationError: ukuran memory tidak boleh negatif");
                }
                if (static_cast<std::size_t>(instruction.operand) > memory.size()) {
                    memory.resize(static_cast<std::size_t>(instruction.operand), 0);
                }
                ++ip;
                break;
            }

            case OpCode::LIT: {
                push(instruction.operand);
                ++ip;
                break;
            }

            case OpCode::LOD: {
                RuntimeProtection::checkMemoryAddress(instruction.operand, memory.size(), "LOD");
                push(memory[static_cast<std::size_t>(instruction.operand)]);
                ++ip;
                break;
            }

            // ... instruksi lainnya (STO, JMP, JPC, dll)

```

Cuplikan Kode: Contoh komputasi matematis yang diamankan oleh modul `RuntimeProtection` (misalnya pada penanganan pembagian dan *overflow*):

```
int RuntimeProtection::safeDiv(int left, int right) {  
    if (right == 0) {  
        throw RuntimeError("DivisionByZeroError: pembagian dengan nol tidak diperbolehkan");  
    }  
    if (left == INT_MIN_LIMIT && right == -1) {  
        throw RuntimeError("OverflowError: operasi DIV melewati batas integer");  
    }  
    return left / right;  
}
```

BAB 3 : Pengujian

Input	Output
Case 1	
<pre>program ArrayTest; const PI = 3.14; var numbers: array[1..5] of real; i: integer; begin for i := 5 downto 1 do begin numbers[i] := i * PI; end; end; end. program RecordTest; type Point = record x, y: integer; end; var p1: Point; begin p1.x := 10; p1.y := 20; { Menguji apakah lexer bingung antara . (period) dan .. (range) } writeln('Coordinates: ', p1.x, '.', p1.y); end.</pre>	<pre>0 INT 0 1000 1 LIT 0 5 2 STO 0 6 3 LOD 0 6 4 LIT 0 1 5 OPR 0 12 6 JPC 0 23 7 LOD 0 6 8 LOD 0 0 9 OPR 0 4 10 LIT 0 5 11 LOD 0 6 12 LIT 0 0 13 OPR 0 3 14 LIT 0 0 15 OPR 0 4 16 OPR 0 2 17 STOI 0 0 18 LOD 0 6 19 LIT 0 1 20 OPR 0 2 21 STO 0 6 22 JMP 0 3 23 RET 0 0</pre>


```

--- DECORATED AST ---
└─ BlockNode
  └─ BlockNode
    └─ VarDecl: numbers : arraysy --> [Type: arraysy | Scope: 0]
    └─ BlockNode
      └─ BlockNode
        └─ ForStatement(var: i) --> [Type: void | Scope: 0]
          └─ Start:
            └─ NumberNode(5)
          └─ End:
            └─ NumberNode(1)
          └─ Body:
            └─ BlockNode
              └─ BlockNode
                └─ Assign --> [Type: void | Scope: 0]
                  └─ ArrayAccess(array: numbers) --> [Type: void | Scope: 0]
                    └─ VariableNode(i)
                  └─ BinaryExpr: '*' --> [Type: void | Scope: 0]
                    └─ VariableNode(i)
                    └─ VariableNode(PI)

```

```

--- Milestone 4 Vulnerability Protection Test ---
[PASS] TC-01 Stack underflow pada OPR ADD -> StackUnderflowError: operasi membutuhkan 1 nilai, tetapi stack hanya berisi 0 nilai
[PASS] TC-02 Invalid jump target -> InvalidJumpTargetError: target jump 99 tidak ada pada intermediate code berukuran 1
[PASS] TC-03 Memory out-of-bounds pada LOD -> MemoryAccessError: instruksi LOD mencoba mengakses address 10, tetapi ukuran memory hanya 5
[PASS] TC-04 Numerical overflow pada ADD -> OverflowError: operasi ADD melewati batas maksimum integer
[PASS] TC-05 Division by zero -> DivisionByZeroError: pembagian dengan nol tidak diperbolehkan
[PASS] TC-06 Array index out-of-bounds helper -> IndexOutOfBoundsExpection: indeks 10 di luar batas array numbers [1..5]
[PASS] TC-07 Program valid: y := 10 + 5; writeln(y) -> Output='15'
', expected='15'

Summary: 7/7 test passed

```

Case 2

```

program Hello;

var
  a, b, i: integer;

function AddTen(x: integer): integer;
begin
  AddTen := x + 10;
end;

begin
  a := 5;
  b := AddTen(a);
  writeln('Result = ', b);
end.

```

```

0 INT 0 1000
1 JMP 0 4
2 INT 0 15
3 RET 0 0
4 LIT 0 5
5 STO 0 5
6 LIT 0 0
7 STO 0 6
8 LOD 0 6
9 OPR 0 14
10 RET 0 0

```

```

--- DECORATED AST ---
└─ BlockNode
    └─ BlockNode
        └─ VarDecl: a : integer --> [Type: integer | Scope: 0]
        └─ VarDecl: comma : integer --> [Type: integer | Scope: 0]
        └─ VarDecl: b : integer --> [Type: integer | Scope: 0]
        └─ VarDecl: comma : integer --> [Type: integer | Scope: 0]
        └─ VarDecl: i : integer --> [Type: integer | Scope: 0]
        └─ SubprogramDecl: AddTen --> [Type: void | Scope: 0]
            └─ [NULL BLOCK]
    └─ BlockNode
        └─ BlockNode
            └─ Assign --> [Type: void | Scope: 0]
                └─ VariableNode(a)
                    └─ NumberNode(5)
            └─ Assign --> [Type: void | Scope: 0]
                └─ VariableNode(b)
                    └─ CallNode(func: 'AddTen') --> [Type: void | Scope: 0]
                        └─ VariableNode(a)
            └─ CallNode(func: 'writeln') --> [Type: void | Scope: 0]
                └─ StringNode('Result = ')
                    └─ VariableNode(b)

```

```

--- Milestone 4 Vulnerability Protection Test ---
[PASS] TC-01 Stack underflow pada OPR ADD -> StackUnderflowError: operasi membutuhkan 1 nilai, tetapi stack hanya berisi 0 nilai
[PASS] TC-02 Invalid jump target -> InvalidJumpTargetError: target jump 99 tidak ada pada intermediate code berukuran 1
[PASS] TC-03 Memory out-of-bounds pada LOD -> MemoryAccessError: instruksi LOD mencoba mengakses address 10, tetapi ukuran memory hanya 5
[PASS] TC-04 Numerical overflow pada ADD -> OverflowError: operasi ADD melewati batas maksimum integer
[PASS] TC-05 Division by zero -> DivisionByZeroError: pembagian dengan nol tidak diperbolehkan
[PASS] TC-06 Array index out-of-bounds helper -> IndexOutOfBoundsException: indeks 10 di luar batas array numbers [1..5]
[PASS] TC-07 Program valid: y := 10 + 5; writeln(y) -> Output='15'
', expected='15'
Summary: 7/7 test passed

```

Case 3

```

program TypeMismatchAssign;
var
  angka: integer;
  teks: string;
begin
  teks := 'Kompilator Arion';
  angka := teks;
end.

```

```

0 INT 0 1000
1 JMP 0 4
2 INT 0 15
3 RET 0 0
4 LIT 0 5
5 ST0 0 5
6 LIT 0 0
7 ST0 0 6
8 LOD 0 6
9 OPR 0 14
10 RET 0 0

```

Semantic Error: Type mismatch dalam assignment. Tidak dapat menetapkan nilai ke target.

Analisis semantik gagal

Case 4

```

program ValidProgram;
var
  a, b: integer;
  res: real;
begin
  a := 10;
  b := 20;
  res := a + b;
  writeln('Hasilnya adalah');
end.

```

```

0 INT 0 1000
1 LIT 0 10
2 STO 0 5
3 LIT 0 20
4 STO 0 6
5 LOD 0 5
6 LOD 0 6
7 OPR 0 2
8 STO 0 7
9 RET 0 0

```

```

--- DECORATED AST ---
└─ BlockNode
  └─ BlockNode
    └─ VarDecl: a : integer --> [Type: integer | Scope: 0]
    └─ VarDecl: comma : integer --> [Type: integer | Scope: 0]
    └─ VarDecl: b : integer --> [Type: integer | Scope: 0]
  └─ BlockNode
    └─ BlockNode
      └─ Assign --> [Type: void | Scope: 0]
        └─ VariableNode(a)
          └─ NumberNode(10)
      └─ Assign --> [Type: void | Scope: 0]
        └─ VariableNode(b)
          └─ NumberNode(20)
      └─ Assign --> [Type: void | Scope: 0]
        └─ VariableNode(res)
          └─ BinaryExpr: '+' --> [Type: void | Scope: 0]
            └─ VariableNode(a)
            └─ VariableNode(b)
      └─ CallNode(func: 'writeln') --> [Type: void | Scope: 0]
        └─ StringNode('Hasilnya adalah')

```

Case 5

```

program RecordTest;
type
  Point = record
    x, y: integer;
  end;
var
  p1: Point;
begin
  p1.x := 10;
  p1.y := 20;
  { Menguji apakah lexer bingung antara . (period) dan .. (range) }
  writeln('Coordinates: ', p1.x, ',', p1.y);
end.

```

```

0 INT 0 1000
1 LIT 0 10
2 LIT 0 20
3 LOD 0 5
4 OPR 0 13
5 LOD 0 5
6 OPR 0 14
7 RET 0 0

```

```

--- DECORATED AST ---
└─ BlockNode
  └─ BlockNode
    └─ VarDecl: p1 : Point --> [Type: Point | Scope: 0]
  └─ BlockNode
    └─ BlockNode
      └─ Assign --> [Type: void | Scope: 0]
        └─ FieldAccess: p1.x --> [Type: void | Scope: 0]
          └─ NumberNode(10)
      └─ Assign --> [Type: void | Scope: 0]
        └─ FieldAccess: p1.y --> [Type: void | Scope: 0]
          └─ NumberNode(20)
      └─ CallNode(func: 'writeln') --> [Type: void | Scope: 0]
        └─ StringNode('Coordinates: ')
        └─ FieldAccess: p1.x --> [Type: void | Scope: 0]
        └─ StringNode(',')
        └─ FieldAccess: p1.y --> [Type: void | Scope: 0]

```

```

--- Milestone 4 Vulnerability Protection Test ---
[PASS] TC-01 Stack underflow pada OPR ADD -> StackUnderflowError: operasi membutuhkan 1 nilai, tetapi stack hanya berisi 0 nilai
[PASS] TC-02 Invalid jump target -> InvalidJumpTargetError: target jump 99 tidak ada pada intermediate code berukuran 1
[PASS] TC-03 Memory out-of-bounds pada LOD -> MemoryAccessError: instruksi LOD mencoba mengakses address 10, tetapi ukuran memory hanya 5
[PASS] TC-04 Numerical overflow pada ADD -> OverflowError: operasi ADD melewati batas maksimum integer
[PASS] TC-05 Division by zero -> DivisionByZeroError: pembagian dengan nol tidak diperbolehkan
[PASS] TC-06 Array index out-of-bounds helper -> IndexOutOfBoundsExpection: indeks 10 di luar batas array numbers [1..5]
[PASS] TC-07 Program valid: y := 10 + 5; writeln(y) -> Output='15'
', expected='15'

Summary: 7/7 test passed

```

BAB 4 : Kesimpulan dan Saran

4.1. Kesimpulan

Program berhasil mengonversi parsing tree menjadi intermediate code terstruktur dan mengeksekusi instruksi tersebut menggunakan mekanisme interpreter. Tabel simbol pada C++ terbukti dapat mengelola memori dan status variabel secara dinamis selama *runtime*, serta mengkalkulasi ekspresi aritmatika dan alur kontrol logika secara akurat.

4.2. Saran

Skema intermediate code dapat dikembangkan lebih lanjut untuk mendukung tahap optimasi dasar, seperti *constant folding* (evaluasi konstanta statis sebelum *runtime*) dan *dead code elimination*. Desain arsitektur interpreter juga memerlukan ekspansi berupa *call stack* memori untuk dapat mengeksekusi deklarasi fungsi kustom dan sistem rekursi.

Lampiran

- Link Repository Github = <https://github.com/kalkabena/XYZ-Tubes-IF2224-2026.git>
- Tabel Pembagian Tugas:

NIM	Nama	Pembagian Tugas	Persentase Kontribusi
13524114	Mirza Tsabita Wafa'ana	Mengerjakan kode bagian vulnerability testing dan mengerjakan laporan	33%
13524119	Nathanael Shane Bennet		0%
13524130	Faris Wirakusuma Triawan	Mengerjakan kode bagian ICG dan VirtualMachine dan mengerjakan laporan	33%
13524144	Jonathan Harijadi	Mengerjakan kode bagian StackMachine dan mengerjakan laporan	33%

Referensi

“Compilers - Principles, Techniques, and Tools (2006)”

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

Guidebook Konvensi Intermediate Code

 Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebook Konvensi Intermedi...

[1] R. Nystrom, “A Virtual Machine,” Crafting Interpreters, 2021. [Online]. Available: <https://craftinginterpreters.com/a-virtual-machine.html>. [Accessed: 6-Jun-2026].

[2] R. Nystrom, “Chunks of Bytecode,” Crafting Interpreters, 2021. [Online]. Available: <https://craftinginterpreters.com/chunks-of-bytecode.html>. [Accessed: 6-Jun-2026].